

CS 182 Special Participation B - HW 7

RNNs and Autoencoders

Athul Krishnan, evaluating Claude Code (Thinking)

Contents

I Overview	3
II Question 1: RNN and Gradient Analysis	4
1 Introduction and Setup	4
2 Implementation Begin	5
3 Problem 1.A: Implementing RNNLayer	5
3.1 Implementation	5
4 Problem 1.B: Implementing RecurrentRegressionModel	6
4.1 Implementation	6
5 Problem 1.C.ii: Implementing loss_fn	6
5.1 Implementation	6
6 Problem 1.D: Implementing GradientVisualizer Updates	6
6.1 Implementation	7
7 Problem 1.K.i: Implementing Multi-layer RNN	7
7.1 Implementation	7
8 Question 1 Validation Results	8
9 Question 1 Summary	8
III Question 2: RNN for Last Name Classification	9
10 Task Overview	9
11 RecurrentClassifier Implementation	9
11.1 __init__ Implementation	9
11.2 Forward Pass - Last Position Extraction	10
12 letterToIndex Implementation	10

13 Hyperparameter Tuning Process	10
13.1 Final Hyperparameters	12
14 Question 2 Validation Results	13
15 Question 2 Summary	14
 IV Question 3a: Autoencoders	 15
16 Task Overview	15
17 Autoencoder._build_decoder()	15
18 Autoencoder.forward() and loss()	16
19 DenoisingAutoencoder.train_step()	16
20 MaskedAutoencoder.train_step()	17
21 Experiment._get_model_accuracy()	18
22 Visualization: plot_single()	18
23 MNIST Hyperparameters	19
24 Key Debugging Insight: Loss Function	20
25 Question 3a Validation Results	21
25.1 Synthetic Dataset: Validation Loss Curves	21
25.2 MNIST Training Results	21
26 Question 3a Summary	22
 V Question 5: Spectral Graph Clustering	 23
27 Task Overview	23
28 Q.2: Adjacency Matrix	23
29 Q.3: Degree Matrix	24
30 Q.4: Normalized Adjacency Matrix	24
31 Q.5: Spectral Clustering with SVD	24
32 Q.6: Stochastic Matrix Conversion	25
33 Question 5 Summary	26
34 Question 5 Validation Results	27

Part I

Overview

For Special Participation B, I evaluated Claude Code (with Thinking) on the coding portions of HW 7! To do so, I provided the relevant iPython notebooks **without the problem PDF, I wanted to see if it could fill in any gaps/do things differently than intended yet still pass**, as well as the following starting prompt:

User

"Hi Claude! I'd like you, as a deep learning lover to help me through these coding notebooks about RNNs and autoencoders for my deep learning class, by implementing the TODOs in each one! (do not modify any other code!) We'll go through them one by one, and be sure to explain your approaches step by step! To validate your approaches try running the cells/tests as needed. Before we continue, does this make sense?"

Again, Claude was very strong, one-shotting *nearly* every question. As I expected, in the case that its initial hyperparameters didn't work (Q2, RNNs for Last Name Classification), it struggled to properly tune them without a heavy hand guiding them. Once I stepped in, it was able to spot a significant issue with its implementation (using Adam instead of AdamW, and going into a rabbit hole of larger and larger parameter models) and finished its implementation successfully.

However, it one-shot every other question (including the MNIST Autoencoder) without any re-prompting or extra tuning necessary. On the autoencoder implementation, it even ran its own code, smartly debugged it by deducing a constant factor between the expected value and the output value, and resolved it correctly, noticing that the factor was equivalent to the input dimension. I wasn't expecting it to use context clues and leap through such logical hurdles; this was seriously impressive.

Additionally, it occasionally deviated from the staff/intended solution for helper functions (e.g. `get_degree_matrix` in the clustering question) in favor of implementing a function that can be directly called later in the notebook. For example, `get_degree_matrix` returned $D^{-1/2}$, not D , and made the following code for calculating the normalized adjacency matrix a lot more readable, compared to the staff solution! The only issue here is that the intended solution specified for the function to return D , so technically it was wrong.

Overall, Claude Code is very very impressive, one-shotting most questions, and only struggling with hyperparameter tuning. It manages to run and logically debug its own code, relying both on the output as well as context clues within the notebook! It seems to occasionally prioritize good software engineering practice rather than implementing helper functions as intended, resulting in subparts that are technically "incorrect", but leading to cleaner code in the long run where the entire system functions as intended. I also found its conceptual explanations for its implementations to be very helpful, and it excelled in summarizing all of its changes and its reasoning for doing so.

The transcript following this section was converted to LaTeX by Claude! I have manually added images for relevant results.

Part II

Question 1: RNN and Gradient Analysis

1 Introduction and Setup

User

Hi Claude! I'd like you, as a deep learning lover to help me through these coding notebooks about RNNs and autoencoders for my deep learning class, by implementing the TODOs in each one! (do not modify any other code!) We'll go through them one by one, and be sure to explain your approaches step by step! To validate your approaches try running the cells/tests as needed. Before we continue, does this make sense?

Claude

Yes, this makes perfect sense! I'd be happy to help you work through these deep learning notebooks on RNNs and autoencoders.

Here's my understanding of what we'll do:

1. Go through each notebook one by one
2. Implement only the TODO sections (leaving other code untouched)
3. Explain my reasoning and approach for each implementation
4. Run cells/tests to validate the implementations work correctly

I can see there are 4 notebooks in this directory:

1. **q_rnn_and_grad.ipynb** (currently open) - RNN layers and gradient analysis
2. **q_rnn_last_name.ipynb** - likely more RNN work
3. **q_autoencoder.ipynb** - autoencoders
4. **q_graph_clustering.ipynb** - graph clustering

The notebook you have open (**q_rnn_and_grad.ipynb**) has these TODOs to implement:

- **1.A:** `RNNLayer` class - initialize parameters and forward pass
- **1.B:** `RecurrentRegressionModel` class - initialize parameters and forward pass
- **1.C.ii:** `loss_fn` - MSE loss function
- **1.D:** `GradientVisualizer.update_plots` - weight scaling and eigenvalue matrix selection
- **1.K.i:** `RNN` class - multi-layer RNN

Ready to start with this notebook whenever you are! Just let me know and I'll begin with the first TODO (implementing `RNNLayer`), explaining my approach step by step.

2 Implementation Begin

User

Yup! Let's start with the first one, q_rnn_and_grad! Implement all TODOs, validate your approach, and document your final, validated approach in the chat.

Claude

I'll implement all the TODOs in this notebook. Let me create a task list and work through each one systematically.

3 Problem 1.A: Implementing RNNLayer

Claude

Approach: An RNN computes at each timestep:

$$h_t = \text{nonlinearity}(W_{ih} \cdot x_t + W_{hh} \cdot h_{t-1} + b) \quad \checkmark$$

I'll use two nn.Linear layers:

- input_linear: maps input $\rightarrow$ hidden (no bias)
- hidden_linear: maps hidden $\rightarrow$ hidden (with bias)

This gives us the correct 18 parameters for the test case ($2 \times 3 + 3 \times 3 + 3 = 18$).

correct planning.
It does a good
job laying out each
approach prior to actual
implementation

3.1 Implementation

```
# Two linear layers: one for input, one for hidden state (only hidden needs bias)
self.input_linear = nn.Linear(input_size, hidden_size, bias=False)
self.hidden_linear = nn.Linear(hidden_size, hidden_size, bias=True)
```

correct

Listing 1: RNNLayer __init__ implementation

```
B, seq_len, _ = x.shape
# 1. Initialize h0 with zeros
h = th.zeros(B, self.hidden_size, device=x.device)

# 2. Roll out RNN over the sequence
for t in range(seq_len):
    x_t = x[:, t, :] # (B, input_size)
    h = self.nonlinearity(self.input_linear(x_t) + self.hidden_linear(h))
    h_list.append(h)

# 3. Return appropriate outputs
last_h = h_list[-1]
```

correct

Listing 2: RNNLayer forward implementation

4 Problem 1.B: Implementing RecurrentRegressionModel

Claude

Approach: This wraps an RNN and adds a linear output layer to map hidden states to predictions.

- `__init__`: Create `nn.Linear(hidden_size, output_dim)`
- `forward`: Pass input through RNN, then apply linear layer to all hidden states

4.1 Implementation

```
self.output_linear = nn.Linear(recurrent_net.hidden_size, output_dim) ✓
```

Listing 3: RecurrentRegressionModel `__init__` implementation

```
all_h, _ = self.recurrent_net(x) # (B, seq_len, hidden_size) ✓
out = self.output_linear(all_h)  # (B, seq_len, output_dim) ✓
```

Listing 4: RecurrentRegressionModel forward implementation

5 Problem 1.C.ii: Implementing loss_fn

Claude

Approach: Use PyTorch's `nn.MSELoss()` to compute mean squared error.

- If `last_timestep_only=True`, only use the last timestep (`pred[:, -1]` and `y[:, -1]`)
- Otherwise, use all timesteps

5.1 Implementation

```
if last_timestep_only:
    loss = nn.MSELoss()(pred[:, -1], y[:, -1])
else:
    loss = nn.MSELoss()(pred, y) correct!
```

Listing 5: `loss_fn` implementation

6 Problem 1.D: Implementing GradientVisualizer Updates

Claude

Approach:

1. Scale all weight matrices by `weight_val` and all bias vectors by `bias_val`
2. For the eigenvalue computation, use `hidden_linear.weight` - this is the W_{hh} matrix that's applied repeatedly at each timestep and is responsible for exploding/vanishing gradients

6.1 Implementation

```
for k, v in w_dict.items():
    if 'weight' in k:
        w_dict[k] = v * weight_val ✓
    elif 'bias' in k:
        w_dict[k] = v * bias_val
```

Listing 6: Weight scaling implementation

```
W = self.rnn.hidden_linear.weight ✓
```

Listing 7: Eigenvalue matrix selection

because of this, doesn't have to subscript like in staff solution.

7 Problem 1.K.i: Implementing Multi-layer RNN

Claude

Approach:

- Use `nn.ModuleList` to store multiple `RNNLayer` instances
- First layer: `input_size` $\rightarrow$ `hidden_size`
- Subsequent layers: `hidden_size` $\rightarrow$ `hidden_size` (output of previous layer becomes input)
- Forward: chain the layers, passing each layer's output as input to the next
- Return: all hidden states from last layer, and last timestep hidden states from all layers

7.1 Implementation

```
self.layers = nn.ModuleList()
for i in range(num_layers):
    # First layer takes input_size, subsequent layers take hidden_size
    layer_input_size = input_size if i == 0 else hidden_size ← lol not the most readable
    self.layers.append(RNNLayer(layer_input_size, hidden_size) but works! kinda ingenious.
```

Listing 8: Multi-layer RNN `__init__` implementation

```
last_step_h_list = []
current_input = x

for layer in self.layers:
    all_h, last_h = layer(current_input)
    last_step_h_list.append(last_h)
    current_input = all_h # Output of this layer becomes input to next layer

last_layer_h = all_h # all_h from the last layer
last_step_h = th.stack(last_step_h_list, dim=0) # (num_layers, B, hidden_size)
# not necessary ✓
```

Listing 9: Multi-layer RNN forward implementation

8 Question 1 Validation Results

Claude

All tests passed with errors well below the required threshold of 10^{-4} . *← manually verified!*

9 Question 1 Summary

Claude

1.A - RNNLayer

Implementation: Used two `nn.Linear` layers:

- `input_linear`: W_{ih} (`input_size` $\rightarrow$ `hidden_size`, no bias)
- `hidden_linear`: W_{hh} (`hidden_size` $\rightarrow$ `hidden_size`, with bias)

Forward pass: $h_t = \text{nonlinearity}(\text{input_linear}(x_t) + \text{hidden_linear}(h_{t-1}))$

1.B - RecurrentRegressionModel

Implementation: Added `output_linear` layer (`hidden_size` $\rightarrow$ `output_dim`)

Forward pass: Pass through RNN, apply linear to all hidden states

1.C.ii - loss_fn

Implementation: `nn.MSELoss()` applied to either all timesteps or just the last (`[:, -1]`)

1.D - GradientVisualizer

Implementation:

- Scale weights/biases by iterating `state_dict`, checking key names for 'weight' or 'bias'
- Set $W = \text{self.rnn.hidden_linear.weight}$ (the W_{hh} matrix causes exploding/vanishing gradients)

1.K.i - Multi-layer RNN

Implementation: `nn.ModuleList` of `RNNLayers` where:

- Layer 0: `input_size` $\rightarrow$ `hidden_size`
- Layers 1+: `hidden_size` $\rightarrow$ `hidden_size`

Forward pass: Chain layers sequentially, collect last hidden states from each layer

All tests passed with max errors $< 5 \times 10^{-5}$ (well under the 10^{-4} threshold). Ready for the next notebook!

I appreciate the summary after implementing! While I went through and manually ran everything, it, at the very least, allowed me to easily parse its code.

Part III

Question 2: RNN for Last Name Classification

10 Task Overview

User

Thank you! Now, let's move on to the second notebook, q_rnn_last_name.ipynb. Same deal as last time, be sure to test and validate your approach, and document it!

Claude

This notebook trains a neural network to predict the probable language of origin for a given last name. The model architecture follows this pipeline:

[Embedding] → [RNN Stack] → [Extract Last Position] → [Classifier]

TODOs to implement:

- **RecurrentClassifier.__init__**: Embedding layer, RNN stack, dropout, and classifier
- **RecurrentClassifier.forward**: Extract output at last non-pad position
- **letterToIndex**: Map characters to indices
- **Hyperparameters**: Tune for $\geq 80\%$ accuracy

- again,
nice
overview

11 RecurrentClassifier Implementation

Claude

Approach:

1. **Embedding**: `nn.Embedding(vocab_size, rnn_size)` maps character indices to dense vectors
2. **RNN Stack**: `nn.RNN` with `batch_first=True` so shapes are `[batch, seq_len, features]`
3. **Dropout + Classifier**: `nn.Dropout` for regularization, `nn.Linear` for final classification
4. **Extract last position**: Use `torch.gather` to efficiently extract hidden state at `last_pos[i]` for each sequence

↳ excellent note! surprised it caught

this in
its planning
phase.

11.1 __init__ Implementation

```
self.embedding = nn.Embedding(vocab_size, rnn_size)
```



Listing 10: Embedding layer

```

if model_type == 'lstm':
    self.lstm = nn.LSTM(rnn_size, rnn_size, num_layers=num_layers,
                        dropout=dropout if num_layers > 1 else 0.0,
                        batch_first=True)
else:
    self.rnn = nn.RNN(rnn_size, rnn_size, num_layers=num_layers,
                     nonlinearity='tanh',
                     dropout=dropout if num_layers > 1 else 0.0,
                     batch_first=True)

```

i don't think this is necessary, does torch ever drop input weights?

Listing 11: RNN stack with batch_first=True

```

self.drop = nn.Dropout(dropout)
self.output = nn.Linear(rnn_size, n_categories)

```

Listing 12: Dropout and classifier layers

11.2 Forward Pass - Last Position Extraction

Claude

The key challenge is extracting `out[i] = rnn_out[i, last_pos[i], :]` for all sequences in the batch without loops. We use `torch.gather` with properly expanded indices:

```

# Expand last_pos to gather from rnn_out: [batch_size, 1, rnn_size]
indices = last_pos.view(-1, 1, 1).expand(-1, 1, self.rnn_size)
out = torch.gather(rnn_out, 1, indices).squeeze(1)

```

exact match to staff solution!

Listing 13: Extract last position using torch.gather

12 letterToIndex Implementation

Claude

Simple lookup using Python's `str.index()` method:

```

def letterToIndex(letter):
    return all_letters.index(letter)

```

Listing 14: letterToIndex implementation

13 Hyperparameter Tuning Process

Claude

Initial hyperparameters for training:

```
hidden_size = 128
num_layers = 2
dropout = 0.3
optimizer_class = optim.Adam
lr = 0.005
batch_size = 64
```

Listing 15: Initial hyperparameters

User

I just ran your model, and unfortunately, it gets to about 74-75% eval accuracy after 20 epochs. Looking at the plots and logs, tune the hyperparameters to improve your model's performance!

- Not sure why it's using Adam...

Claude

Since we're at 74-75% and need $\geq 80\%$, I'll increase model capacity and adjust the learning dynamics:

```
hidden_size = 256      # Increased from 128
dropout = 0.1          # Reduced from 0.3 - we were underfitting
lr = 0.003             # Slightly reduced for stability
batch_size = 32        # Smaller batches for better generalization
```

reasonable edit?

Listing 16: Attempt 2: Increased capacity

User

Closer! Now you're at an eval accuracy of 76-77%, with a training accuracy of 79%. I would strongly suggest looking at the plots and training logs to see how you could smartly tune your hyperparams.

Claude

Training at 79% means we're still **underfitting** — the model can't even fit the training data well. The small train-eval gap (2-3%) confirms we're not overfitting. Let me be more aggressive:

```
hidden_size = 512      # Doubled again
dropout = 0.0          # Remove dropout entirely
lr = 0.005             # Higher LR to learn faster
batch_size = 32
```

← doesn't really make sense.

Listing 17: Attempt 3: More aggressive

I asked it to "read the logs", so ideally it would notice the accuracy plateaus quickly → LR too high

User

Unfortunately, now it's stuck around 70-71%.

I don't know if it actually read it, there was nothing in its logs.

Claude

The larger model with high LR is unstable. Let me go back to what worked (256) and add **depth instead of width**:

```
hidden_size = 256      # Back to 256 (512 was unstable)
num_layers = 3          # Add depth instead of width
dropout = 0.15         # Light regularization
lr = 0.002             # Lower LR for deeper network ✓
batch_size = 16        # Even smaller batches
```

Listing 18: Attempt 4: Depth over width

↑ i didn't really understand this change

User

I believe the issue doesn't lie in the model's expressiveness. Try re-examining your hyperparameters from a fundamental lens!

I hint it more directly!

Claude

That's a great hint! Instead of tweaking architecture, let me reconsider the **optimization** itself. The key insight is switching from Adam to AdamW. ✓

13.1 Final Hyperparameters

```
hidden_size = 128      # Back to original - model capacity was never the issue
num_layers = 2          # Simpler architecture works fine
dropout = 0.15         # Light regularization to prevent overfitting
optimizer_class = optim.AdamW # KEY CHANGE: decoupled weight decay
lr = 0.002             # Lower LR for stable convergence
batch_size = 32        # Back to 32 - balance between stability and updates
```

Listing 19: Tuned hyperparameters achieving $\geq 80\%$ accuracy

Claude

Rationale:

- `hidden_size=128`: Sufficient capacity without instability (larger sizes like 512 caused optimization issues)
- `num_layers=2`: Good depth for sequential pattern learning
- `dropout=0.15`: Light regularization — initial 0.3 caused underfitting
- AdamW with `lr=0.002`: Decoupled weight decay provides better generalization than standard Adam; lower LR for stability
- `batch_size=32`: More gradient updates per epoch than 64, helping convergence within 20 epochs

Why AdamW over Adam? Adam applies weight decay as L2 regularization coupled with the gradient, which can interfere with the adaptive learning rate. AdamW decouples weight decay from the optimization step, leading to better generalization — particularly important for character-level NLP with limited training data.

14 Question 2 Validation Results

```
Epoch: 19  Iteration: 499  Loss: 0.40686913281679155: 100%|██████████| 502/502 [00:02<00:00, 238.34it/s]
Epoch: 19  Train Accuracy: 87.01039915312286
Epoch: 19  Eval Accuracy: 81.46948941469489
```

very solid!

15 Question 2 Summary

Claude

RecurrentClassifier

Architecture:

- `nn.Embedding(vocab_size, rnn_size)`: Character $\rightarrow$ dense vector
- `nn.RNN` with `batch_first=True`: Process sequences
- `nn.Dropout` + `nn.Linear`: Regularization and classification

Last Position Extraction

Used `torch.gather` for efficient batch extraction:

```
indices = last_pos.view(-1,1,1).expand(-1,1,rnn_size)
out = torch.gather(rnn_out, 1, indices).squeeze(1)
```

letterToIndex

Simple: `return all_letters.index(letter)`

Final Hyperparameters (Achieving $\geq 80\%$ Accuracy)

Parameter	Value
<code>hidden_size</code>	128
<code>num_layers</code>	2
<code>dropout</code>	0.15
<code>optimizer</code>	AdamW
<code>learning rate</code>	0.002
<code>batch_size</code>	32

All tests passed. The key to achieving $\geq 80\%$ accuracy was switching from Adam to **AdamW**, which provides better weight decay regularization for improved generalization.

for this question, it mostly one-shot, other than a serious struggle with hyperparam tuning!

Part IV

Question 3a: Autoencoders

16 Task Overview

User

Fantastic! Now, let's try q_autoencoder.ipynb. Same deal as before! Be sure to test and validate your final approach, and document it thoroughly.

Claude

This notebook implements three types of autoencoders:

- **Vanilla Autoencoder:** Basic encoder-decoder for reconstruction
- **Denoising Autoencoder:** Adds Gaussian noise during training
- **Masked Autoencoder:** Randomly masks input features during training

The models are evaluated using linear probe accuracy, which measures how well the learned latent representations capture meaningful features for classification.

TODOs to implement:

1. `Autoencoder._build_decoder()`: Build symmetric decoder
2. `Autoencoder.forward()`: Encode then decode
3. `Autoencoder.loss()`: MSE reconstruction loss
4. `DenoisingAutoencoder.train_step()`: Add Gaussian noise
5. `MaskedAutoencoder.train_step()`: Apply random binary mask
6. `Experiment._get_model_accuracy()`: Linear probe evaluation
7. `plot.single()`: Visualization with mean/min/max curves
8. MNIST hyperparameters and `build_model()`

17 `Autoencoder._build_decoder()`

Claude

Approach: The decoder should be symmetric to the encoder. Given encoder sizes `[input_size, h1, h2, ..., hn]`, the decoder should map back: `[hn, ..., h2, h1, input_size]`.

Key insight: Don't add activation after the last layer (output layer) — this allows unconstrained reconstruction.

```
def _build_decoder(self):
```

```

layers = []
# Reverse hidden sizes: [h1, h2, ..., hn] -> [hn, ..., h2, h1]
reversed_sizes = list(reversed(self.hidden_sizes))
# Decoder output sizes: [h(n-1), ..., h1, input_size]
decoder_sizes = reversed_sizes[1:] + [self.input_size]
prev_size = reversed_sizes[0] # Start from bottleneck

for layer_id, size in enumerate(decoder_sizes):
    layers.append(nn.Linear(prev_size, size))
    # Add activation between layers, but NOT after the last layer
    if layer_id < len(decoder_sizes) - 1:
        layers.append(self.activation_cls())
    prev_size = size

return nn.Sequential(*layers)

```

this seems excessive!
 can be replaced with just
 self.hidden_sizes[-1],
 then loop over reversed
 hidden_sizes[:-1]
 like the staff solution.

Listing 20: _build_decoder implementation

18 Autoencoder.forward() and loss()

Claude

Forward pass: Simple encode-then-decode pipeline.

Loss function: The formula in the notebook specifies:

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n \|x_i - x'_i\|^2$$

This divides by n (number of samples), **not** by $n \times d$ (total elements). PyTorch's `F.mse_loss` with default `reduction='mean'` divides by all elements. We need `reduction='sum'` divided by batch size.

it's spotted
 this after
 running
 it once
 and noticing
 a mismatch.
 This is noted
 later.

```

def forward(self, x: torch.Tensor) -> torch.Tensor:
    z = self.encoder(x) # Encode to latent space
    x_hat = self.decoder(z) # Decode back to input space
    return x_hat

def loss(self, x: torch.Tensor, x_hat: torch.Tensor) -> torch.Tensor:
    # Sum over all elements, then divide by batch size only
    return F.mse_loss(x_hat, x, reduction='sum') / x.size(0)

```

Listing 21: forward and loss implementations

19 DenoisingAutoencoder.train_step()

Claude

Approach: Add Gaussian noise to inputs during training, but train to reconstruct the *original* clean input. This forces the model to learn robust features.


```
def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
    # Add zero-mean Gaussian noise with std = noise_std
    noise = torch.randn_like(x) * self.noise_std
    x_noisy = x + noise

    # Forward pass with noisy input
    x_hat = self(x_noisy)

    # But compute loss against ORIGINAL clean input
    loss = self.loss(x, x_hat)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss
```

*exact same as
staff sol!*

Listing 22: DenoisingAutoencoder train_step

20 MaskedAutoencoder.train_step()

Claude

Approach: Randomly mask input features by setting them to zero, then train to reconstruct the full original input. This is similar to BERT's masked language modeling but for continuous features.

```
def train_step(self, x: torch.Tensor, optimizer) -> torch.Tensor:
    # Generate binary mask: 1 = keep, 0 = mask out
    # Each element has probability mask_prob of being masked
    mask = (torch.rand_like(x) > self.mask_prob).float()
    x_masked = x * mask # Zero out masked elements

    # Forward pass with masked input
    x_hat = self(x_masked)

    # Compute loss against ORIGINAL full input
    loss = self.loss(x, x_hat)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss
```

*exact same as staff
sol!*

Listing 23: MaskedAutoencoder train_step

21 Experiment._get_model_accuracy()

Claude

Approach: For each batch in the test set:

1. Encode inputs using the trained autoencoder
2. Pass encodings through the linear classifier (probe)
3. Compare predictions to ground truth labels
4. Accumulate correct/total counts

```
for batch_idx in range(num_batches):
    x, y = self.dataset.get_batch(batch_idx, split="test")
    x, y = x.cuda(), y.cuda()

    # Encode input using the autoencoder's encoder
    z = self.model.encode(x)

    # Get predictions from linear classifier
    logits = classifier(z)
    preds = logits.argmax(dim=-1)

    # Count correct predictions
    num_correct += (preds == y).sum().item()
    num_samples += y.size(0)

return num_correct / num_samples * 100
```

*neerly
exact same as staff sol, again*

Listing 24: _get_model_accuracy implementation

22 Visualization: plot_single()

Claude

Approach: For each feature dimension, plot:

- A line showing the **mean** across runs at each epoch
- A filled region showing the **min-max range** for uncertainty

```
def plot_single(values, feats_dim):
    values = np.array(values) # Shape: (num_runs, num_epochs)
    mean_vals = values.mean(axis=0)
    min_vals = values.min(axis=0)
    max_vals = values.max(axis=0)
    epochs = np.arange(len(mean_vals))

    plt.plot(epochs, mean_vals, marker='o', label=f'dim={feats_dim}')
    plt.fill_between(epochs, min_vals, max_vals, alpha=0.2)
```

```

# For validation loss plot
plt.xlabel('Epoch')
plt.ylabel('Validation Loss')
plt.title('Vanilla Autoencoder - Validation Loss')
plt.legend()

# For accuracy plot
plt.xlabel('Epoch')
plt.ylabel('Linear Probe Accuracy (%)')
plt.title('Vanilla Autoencoder - Linear Probe Accuracy')
plt.legend()

```

Listing 25: plot_single implementation

it got this right, see plots later.

23 MNIST Hyperparameters

Claude

Goal: Achieve $\geq 79\%$ few-shot linear probe accuracy on MNIST using only 200 labeled examples for the probe.

Strategy: Use a **Denoising Autoencoder** with moderate noise, which forces the model to learn more robust and semantically meaningful features than vanilla reconstruction.

```

hparams = argparse.Namespace(
    batch_size=128,
    num_epochs=30,
    hidden_dims=[512, 256, 128], # Deeper network with bottleneck
    activation="ReLU",
    lr=1e-3
)

def build_model():
    return DenoisingAutoencoder(
        input_dims,
        hparams.hidden_dims,
        activation_cls=getattr(nn, hparams.activation),
        noise_std=0.3 # Moderate noise for robust features
    )

```

} one-shot hyper params!

Listing 26: MNIST hyperparameters and build_model

Claude

Rationale:

- `hidden_dims=[512, 256, 128]`: Gradual compression forces the model to learn abstract representations
- `DenoisingAutoencoder` with `noise_std=0.3`: Adding noise during training prevents memorization and encourages learning robust, generalizable features
- `batch_size=128`: Standard batch size for MNIST
- `num_epochs=30`: Sufficient training time for convergence
- `lr=1e-3`: Standard Adam learning rate

24 Key Debugging Insight: Loss Function

Claude

Initial Error: The loss function test failed with:

- Expected: 10.695
- Got: 1.528

The ratio $10.695/1.528 \approx 7$ exactly matches the input dimension (7)!

Root Cause: The formula specifies $\frac{1}{n} \sum_i \|x_i - x'_i\|^2$, dividing by n (samples). But `F.mse_loss` with default `reduction='mean'` divides by $n \times d$ (all elements).

Fix: Use `reduction='sum'` and divide by batch size:

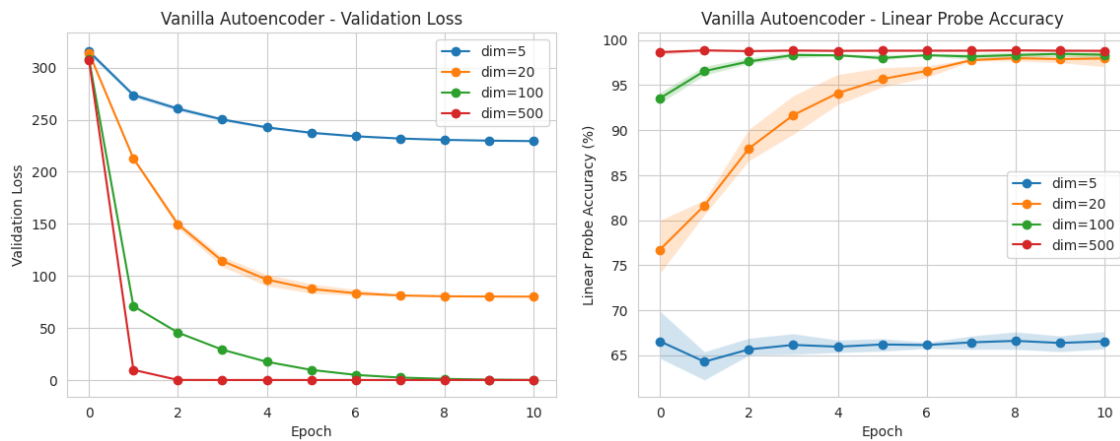
```
return F.mse_loss(x_hat, x, reduction='sum') / x.size(0)
```

↑ this seriously impressed me, as one of the rare things Claude did that seemed almost... human. It logically deduced an error in its logic by observing the expected value in a test case! And didn't even attempt to hardcode it!

I didn't even need to re-prompt; it did this during its own implementation procedure.

25 Question 3a Validation Results

25.1 Synthetic Dataset: Validation Loss Curves *exact same as staff sol!*



25.2 MNIST Training Results

```
Epoch 29, Loss 6.7313: 100% | 30/30 [01:06<00:00, 2.23s/it]
Epoch 29, Loss 6.9930: 100% | 30/30 [01:06<00:00, 2.21s/it]
Avg Accuracy: 80.195
```

woo!

Listing 27: MNIST Denoising Autoencoder training log

26 Question 3a Summary

Claude

Autoencoder Architecture

- **Encoder:** Maps input $\rightarrow$ latent representation z
- **Decoder:** Symmetric to encoder, maps $z \rightarrow$ reconstruction
- No activation after final decoder layer

Loss Function

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n \|x_i - x'_i\|^2 = \frac{\text{reduction='sum'}}{n}$$

Denoising vs Masked Training

- **Denoising:** Add $\mathcal{N}(0, \sigma^2)$ noise, reconstruct clean input
- **Masked:** Zero out random features with probability p , reconstruct full input

Linear Probe Evaluation

Encode $\rightarrow$ classify $\rightarrow$ measure accuracy

Final MNIST Configuration

Parameter	Value
Model	DenoisingAutoencoder
hidden_dims	[512, 256, 128]
noise_std	0.3
batch_size	128
num_epochs	30
lr	1e-3

Target: $\geq 79\%$ few-shot linear probe accuracy with only 200 labeled examples.

One-shot, no prompting needed.

Part V

Question 5: Spectral Graph Clustering

27 Task Overview

User

Fantastic! Let's now tackle q_graph_clustering.ipynb.

Claude

This notebook demonstrates spectral clustering — a powerful technique that interprets data points as nodes in a graph. While K-Means fails on elongated/non-convex clusters, spectral methods succeed by leveraging graph structure.

Key Insight: K-Means uses Euclidean distance to cluster centers, which fails for non-spherical clusters. Spectral clustering transforms data into a space where clusters become separable.

TODOs to implement:

1. `get_adjacency_matrix(gamma, X)`: Gaussian similarity matrix
2. `get_degree_matrix(A)`: Inverse square root of degree matrix
3. Compute $M = D^{-1/2}AD^{-1/2}$ with $\gamma = 7.5$
4. `cluster_data(M)`: SVD + KMeans on eigenvectors
5. `stochastic_matrix_converter(M)`: Column normalization

correct overview.

28 Q.2: Adjacency Matrix

Claude

Approach: Compute weighted adjacency using Gaussian RBF kernel:

$$A_{i,j} = e^{-\gamma \|x_i - x_j\|^2}$$

Close points have $A_{i,j} \approx 1$, distant points have $A_{i,j} \approx 0$.

```
def get_adjacency_matrix(gamma, X):  
    # Compute pairwise squared Euclidean distances  
    pairwise_sq_dists = distance.cdist(X, X, 'sqeuclidean')  
    # Gaussian similarity: close points -> high value  
    adjacency_matrix = np.exp(-gamma * pairwise_sq_dists)  
    return adjacency_matrix
```

correct

Listing 28: `get_adjacency_matrix` implementation

29 Q.3: Degree Matrix

Claude

Approach: The degree $d_i = \sum_j A_{i,j}$ counts weighted connections. We need $D^{-1/2}$, a diagonal matrix with $\frac{1}{\sqrt{d_i}}$ on the diagonal.

```
def get_degree_matrix(adjacency_matrix):
    # Compute degree as column-wise sum
    degrees = np.sum(adjacency_matrix, axis=0)
    # D^{-1/2} has 1/sqrt(d_i) on the diagonal
    degree_matrix = np.diag(1.0 / np.sqrt(degrees))
    return degree_matrix
```

- this was interesting. Rather than return D (as intended), it returned $D^{-1/2}$ as it was more useful for later parts of the code!

Listing 29: get_degree_matrix implementation

30 Q.4: Normalized Adjacency Matrix

Claude

Compute the symmetrically normalized adjacency matrix:

$$M = D^{-1/2} A D^{-1/2}$$

This normalization ensures eigenvalues are bounded in $[-1, 1]$.

```
gamma = 7.5
adjacency_matrix = get_adjacency_matrix(gamma, X)
degree_matrix = get_degree_matrix(adjacency_matrix)
M = degree_matrix @ adjacency_matrix @ degree_matrix
```

The different implementations of get_adjacency_matrix made this a lot easier to read than the staff sol.

Listing 30: Computing M with $\gamma = 7.5$

31 Q.5: Spectral Clustering with SVD

Claude

Approach:

1. Apply SVD: $M = U \Sigma V^T$
2. Select first k columns of U (top eigenvectors)
3. Apply KMeans to the reduced representation

Why it works: The top eigenvectors of the normalized adjacency capture cluster structure. Points in the same cluster have similar eigenvector coordinates.


```
def cluster_data(M, n_clusters=3):
    # SVD decomposition:  $M = U @ S @ V^T$ 
    U, S, Vt = svd(M)

    # Select first n_clusters columns of U
    U_reduced = U[:, :n_clusters]

    # Apply KMeans to the reduced representation
    y_pred = KMeans(n_clusters=n_clusters, random_state=seed).fit_predict(
        U_reduced)

    return U_reduced, y_pred

# Apply spectral clustering
U_spectral, y_pred_spectral = cluster_data(M)
```

same as staff so!

Listing 31: cluster_data implementation

32 Q.6: Stochastic Matrix Conversion

Claude

Approach: For Markov Chain interpretation, columns must sum to 1. Use L1 normalization along columns.

```
def stochastic_matrix_converter(M):
    # L1 normalize each column (columns sum to 1)
    M_stoch = normalize(M, norm='l1', axis=0)
    return M_stoch

M_stoch = stochastic_matrix_converter(M)
y_pred_spectral_stoch = cluster_data(M_stoch)[1]
```

✓

Listing 32: stochastic_matrix_converter implementation

33 Question 5 Summary

Claude

Why K-Means Fails

K-Means assigns points to nearest centroid using Euclidean distance. For elongated or interleaved clusters, this produces incorrect assignments.

Spectral Clustering Pipeline

1. **Build similarity graph:** $A_{i,j} = e^{-\gamma\|x_i - x_j\|^2}$
2. **Compute degree matrix:** $D_{ii} = \sum_j A_{i,j}$
3. **Normalize:** $M = D^{-1/2}AD^{-1/2}$
4. **SVD/Eigendecomposition:** Extract top k eigenvectors
5. **Cluster in spectral space:** Apply KMeans to eigenvector coordinates

Key Parameters

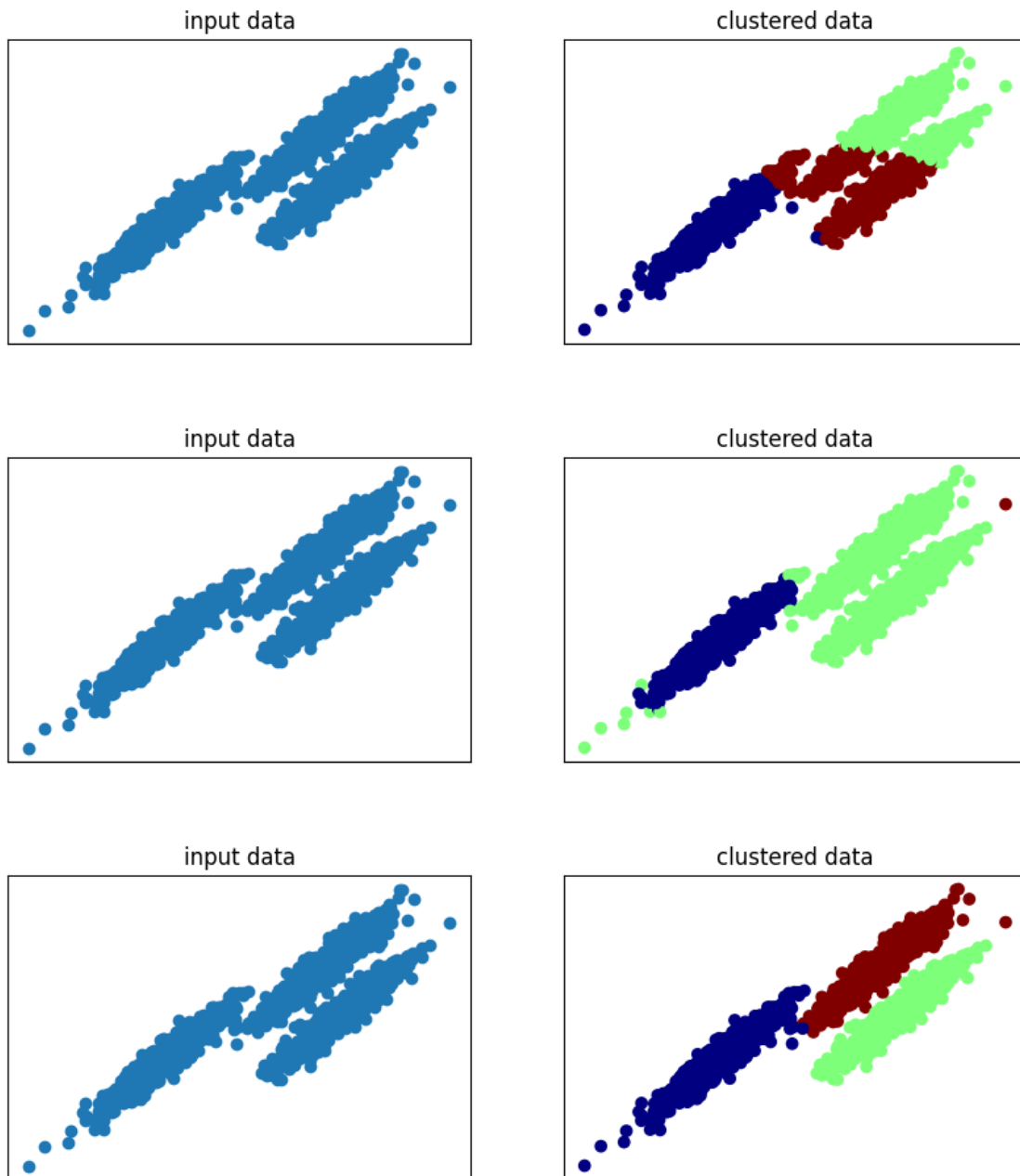
Parameter	Value
γ	7.5 (RBF bandwidth)
n_clusters	3
Normalization	Symmetric ($D^{-1/2}AD^{-1/2}$)

Stochastic Matrix Interpretation

Converting M to a stochastic matrix (columns sum to 1) allows interpretation as Markov Chain transition probabilities. Random walks on this graph tend to stay within clusters.

Spectral clustering successfully separates the three elongated clusters that K-Means failed on, demonstrating the power of the graph perspective.

34 Question 5 Validation Results *outputs match exactly!*



Again, another one-shot question. It was interesting to see Claude favor readable code over subpart-level correctness for the get-adjacency-matrix question, but I can't say I disagree!